CAMUNDA CHAPTER MUNICH

Mastering the distributed transaction problem

What changes when you move from Camunda 7 to Zeebe, and how to keep your data consistent

MIRAGON

Marco Schäck, Miragon GmbH



YOUR SPEAKER

Marco Schäck

Process Development Lead, Miragon

Contact me

marco.schaeck@miragon.io

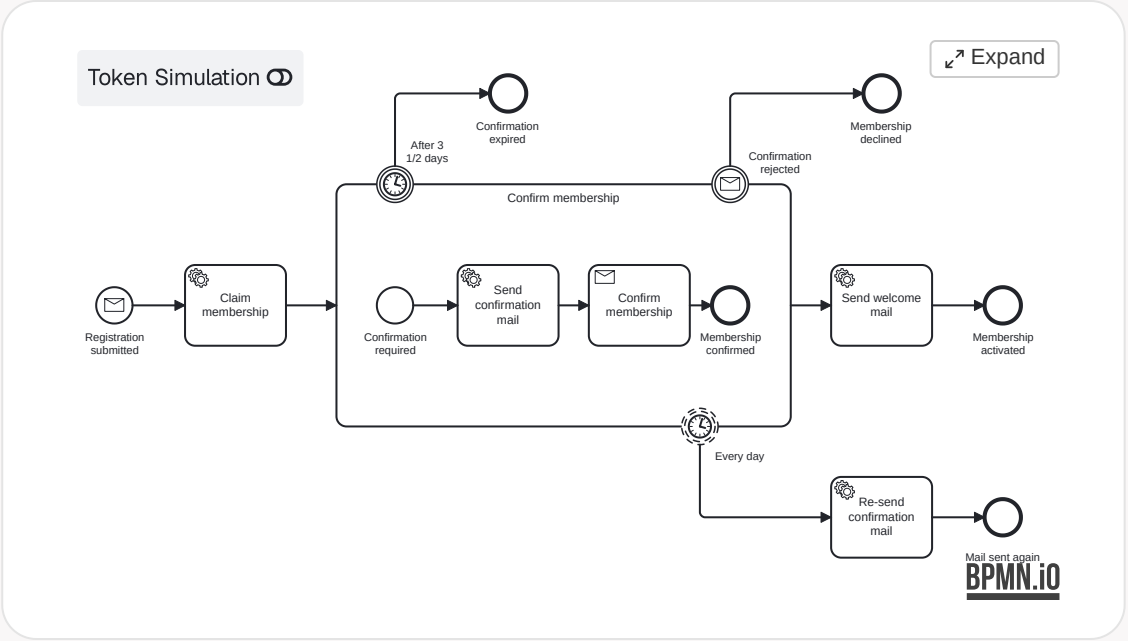
linkedin.com/in/schaeckm

OUR RUNNING EXAMPLE

MiraVelo and its membership program

MiraVelo leases lifestyle bikes for the quarter-life crisis.

- **Perks for members:** discounts, the newsletter, early access to new bikes



THE MIGRATION

From Camunda 7 to Camunda 8

Same process, new engine. And the move went surprisingly well

SUPPORT-TICKET #4270

Registered, but no mail at all

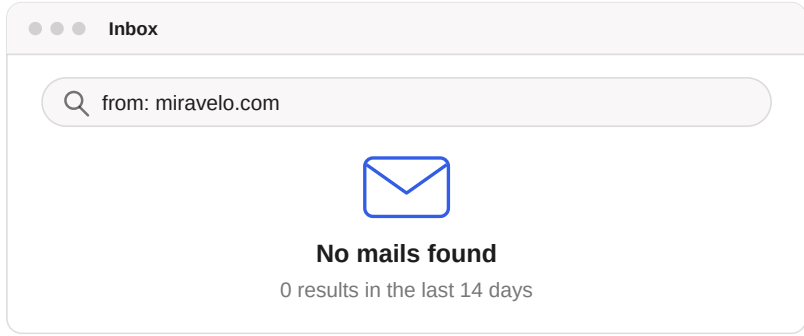
The user says

The database says

The engine says

Registered two weeks ago

Not one mail since then, not even a confirmation mail



SUPPORT-TICKET #4270

Registered, but no mail at all

The user says

The database says

The engine says

No membership

There is no membership request in the database

MEMBERSHIP

```
where email = 'kim@example.com'
```

id	email	status
0 rows		

SUPPORT-TICKET #4270

Registered, but no mail at all

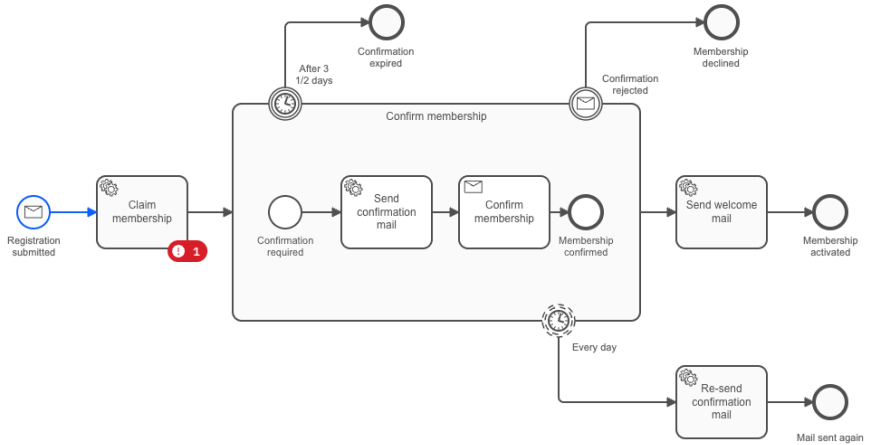
The user says

The database says

The engine says

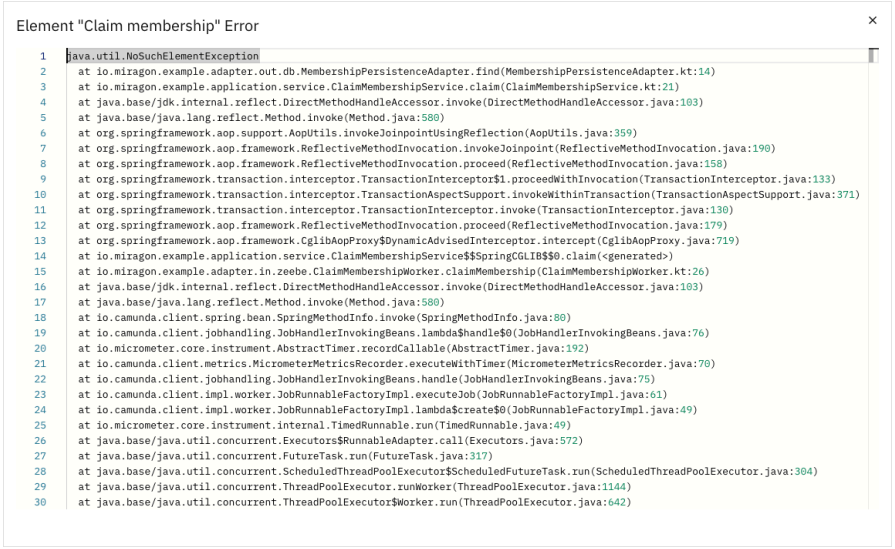
Instance with incident

It is stuck on its very first task



CAMUNDA OPERATE

The worker cannot find the membership



NoSuchElementException: the membership is not in the database.

SUPPORT-TICKET #4271

Welcomed, but not confirmed

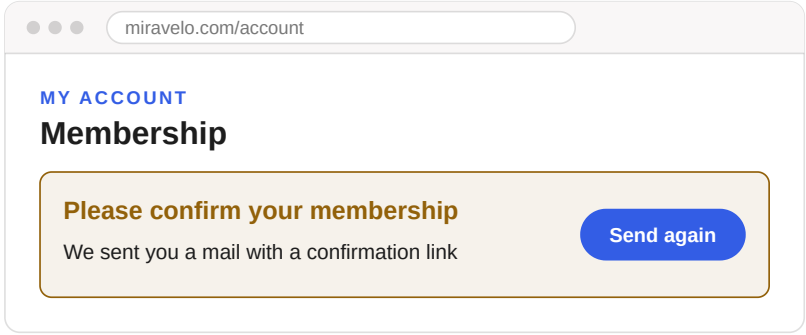
The user says

The database says

The engine says

Confirmed and got the welcome mail

But the account still asks to confirm the membership



SUPPORT-TICKET #4271

Welcomed, but not confirmed

The user says

The database says

The engine says

Not confirmed

The membership is still waiting for confirmation

MEMBERSHIP

id	42
email	kim@example.com
status	WAITING_FOR_CONFIRMATION

SUPPORT-TICKET #4271

Welcomed, but not confirmed

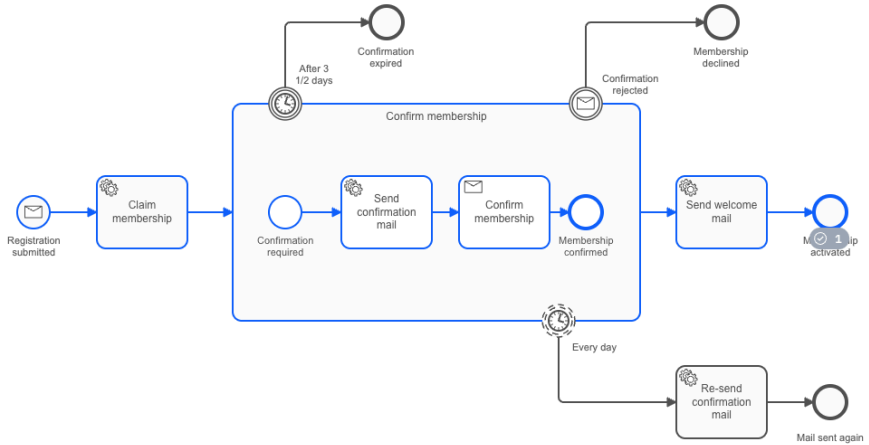
The user says

The database says

The engine says

Instance completed

No incident, every step ran through to the end



TWO TICKETS, ONE MIGRATION

So what has actually changed?

To answer that, we first look back at Camunda 7, and then at what Zeebe does differently

CHAPTER OVERVIEW

What this talk will cover

CHAPTER 1
The Embedded Engine

CHAPTER 3
Distributed Transactions

CHAPTER 5
Idempotency

1 — 2 — 3 — 4 — 5 — 6

The Remote Engine
CHAPTER 2

Exploring solutions
CHAPTER 4

Our Learnings
CHAPTER 6

CHAPTER 1

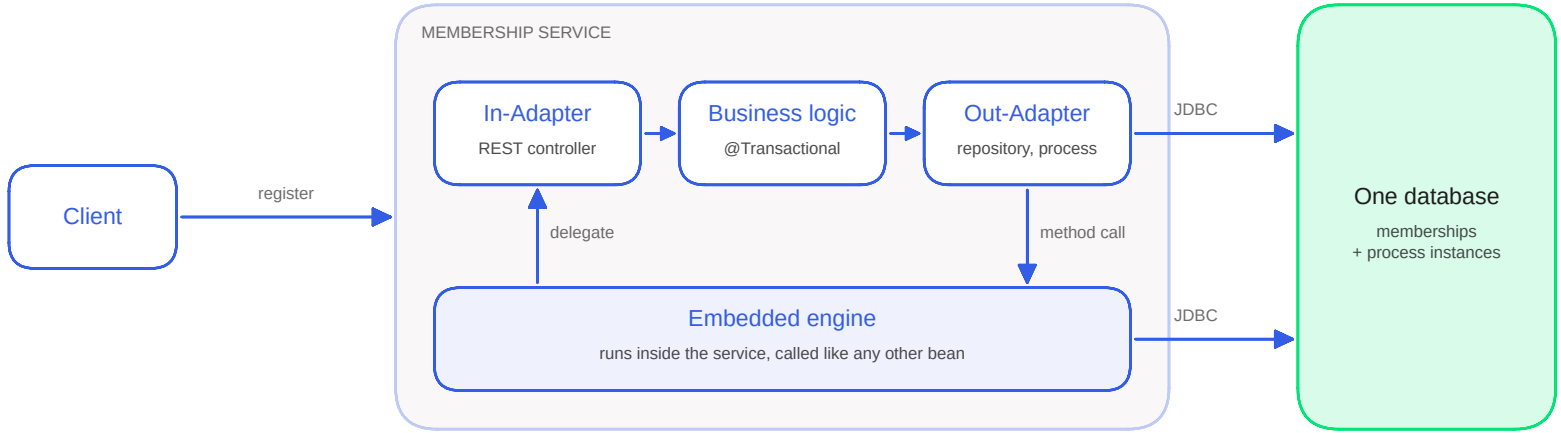
The Embedded Engine

How we knew it

01

CHAPTER 1 · HOW WE BUILT IT

The engine lives inside your service



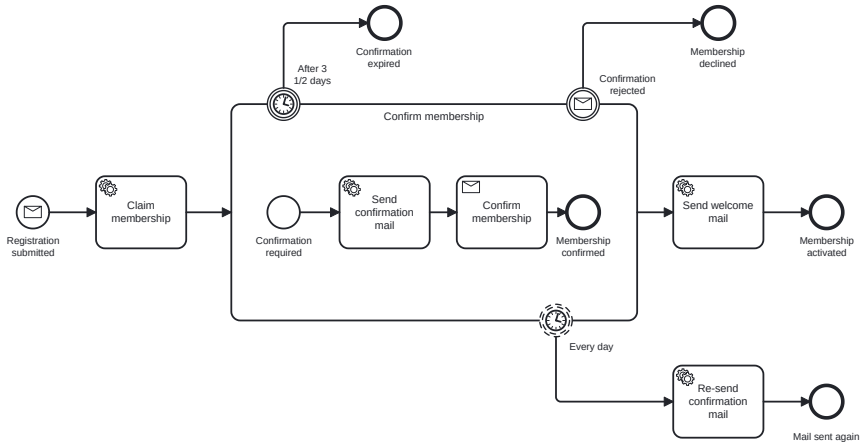
No network in between, and one database for both

CHAPTER 1 · HOW WE BUILT IT

Automating it took little code

One model, three kinds of classes. The engine deploys the model when the service starts

Process | Delegate | Adapter | Service



CHAPTER 1 · HOW WE BUILT IT

Automating it took little code

One model, three kinds of classes. The engine deploys the model when the service starts

Process | **Delegate** | Adapter | Service — KOTLIN

```kotlin
@Component
class SendWelcomeMailDelegate(
    private val useCase: SendWelcomeMailUseCase
) : BaseDelegate() {

    override fun executeTask(execution: DelegateExecution) {
        val membershipId = execution.getVariable("membershipId") as String
        log.debug { "Received task to send welcome mail for membership: $membershipId" }
        useCase.sendWelcomeMail(MembershipId(UUID.fromString(membershipId)))
    }
}
```

CHAPTER 1 · HOW WE BUILT IT

Automating it took little code

One model, three kinds of classes. The engine deploys the model when the service starts

Process | Delegate | **Adapter** | Service — KOTLIN

```kotlin
@Component
class MembershipProcessAdapter(
    private val runtimeService: RuntimeService
) : MembershipProcess {

    override fun submitRegistration(id: MembershipId) {
        val variables = mapOf("membershipId" to id.value.toString())
        runtimeService.startProcessInstanceByMessage(
            "miravelo.registrationSubmitted",
            variables
        )
    }
}
```

CHAPTER 1 · HOW WE BUILT IT

Automating it took little code

One model, three kinds of classes. The engine deploys the model when the service starts

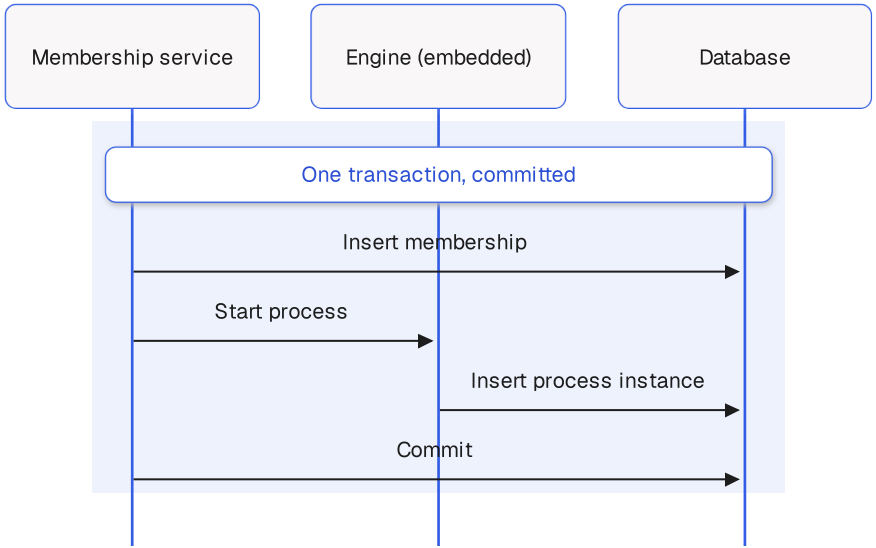
Process | Delegate | Adapter | **Service** — KOTLIN

```kotlin
@Service
@Transactional
class RegisterMembershipService(
    private val repository: MembershipRepository,
    private val processPort: MembershipProcess
) : RegisterMembershipUseCase {

    override fun register(command: RegisterMembershipUseCase.Command): MembershipId {
        val membership = buildMembership(command)
        repository.save(membership)
        processPort.submitRegistration(membership.id)
        return membership.id
    }
}
```

CHAPTER 1 · WHEN IT WORKS

When it works, everyone agrees



The user says

I am a member

The database says

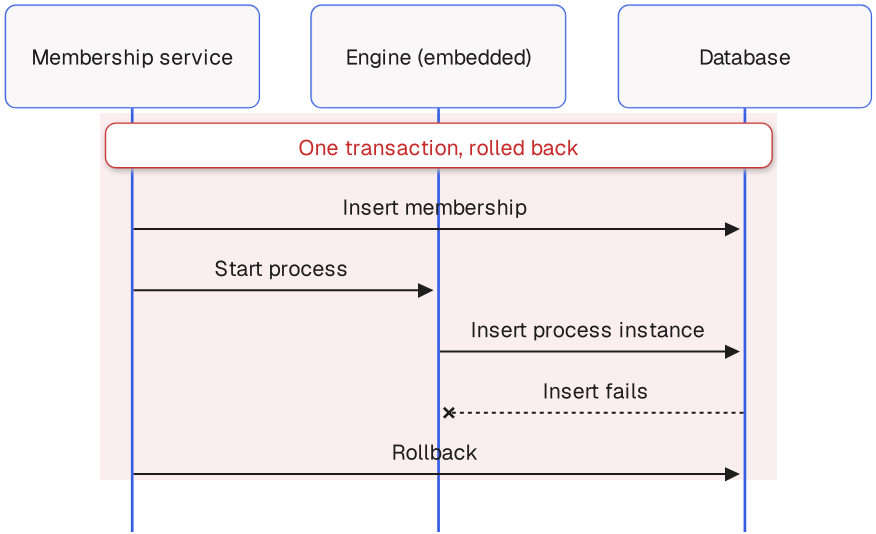
Membership stored

The engine says

Process instance running

CHAPTER 1 · WHEN IT FAILS

When it fails, everyone still agrees



The user says

It did not work, I will try again

The database says

No membership

The engine says

No process instance

OUR SAFEGUARD

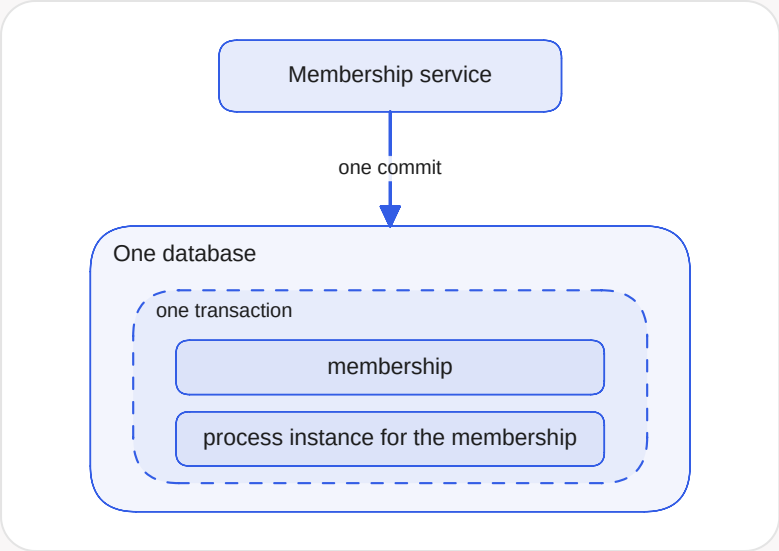
All or nothing

We never wrote that rule. The database enforces it, all we added was `@Transactional`

CHAPTER 1 · WHY IT HOLDS

Our safeguard: ACID

One service, one database, one commit for both

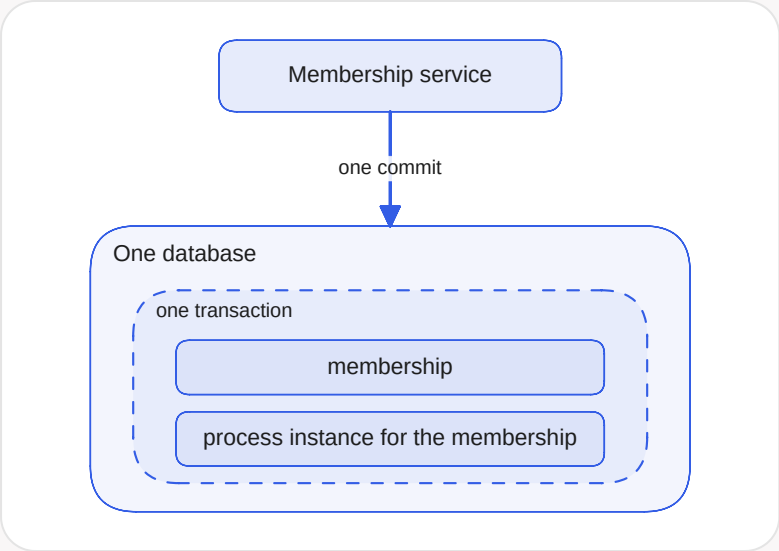


1 **Atomicity:** Both are stored together, or neither is

CHAPTER 1 · WHY IT HOLDS

Our safeguard: ACID

One service, one database, one commit for both

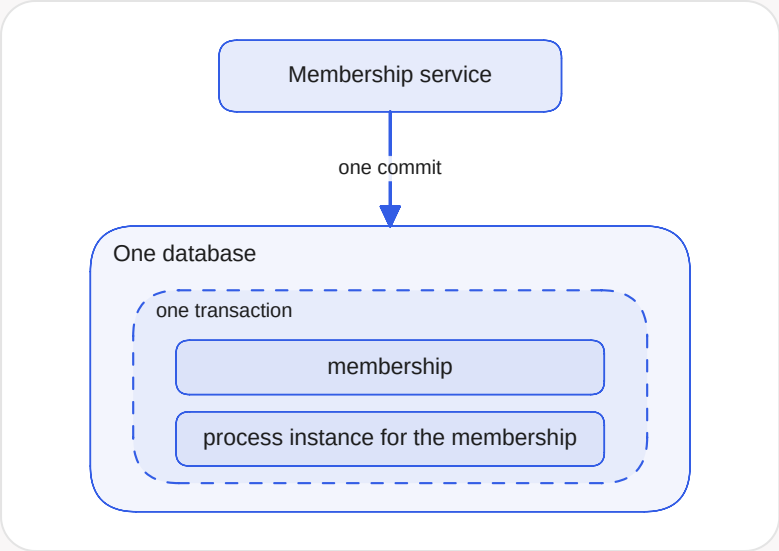


1. **Atomicity:** Both are stored together, or neither is
2. **Consistency:** Only valid states, never a half-finished registration

CHAPTER 1 · WHY IT HOLDS

Our safeguard: ACID

One service, one database, one commit for both

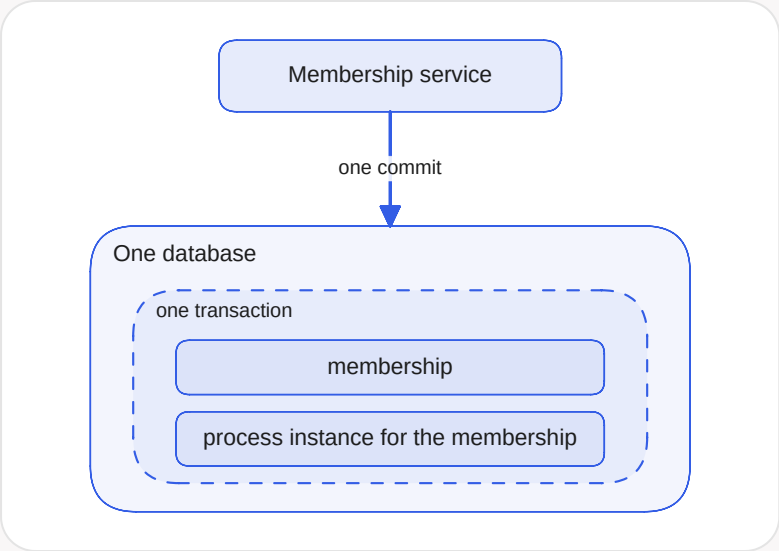


1. **Atomicity:** Both are stored together, or neither is
2. **Consistency:** Only valid states, never a half-finished registration
3. **Isolation:** Nobody sees either of them before the commit

CHAPTER 1 · WHY IT HOLDS

Our safeguard: ACID

One service, one database, one commit for both



1. **Atomicity:** Both are stored together, or neither is
2. **Consistency:** Only valid states, never a half-finished registration
3. **Isolation:** Nobody sees either of them before the commit
4. **Durability:** Once committed, it survives a crash

CHAPTER 2

The Remote Engine

Entering a new world

02

CHAPTER 2

Zeebe, a remote engine

A standalone service that can be operated in the cloud (SaaS) or on-premise. **Why migrating to Zeebe?**

Performance

Higher throughput

Scalability

Horizontally scalable

Cloud-native

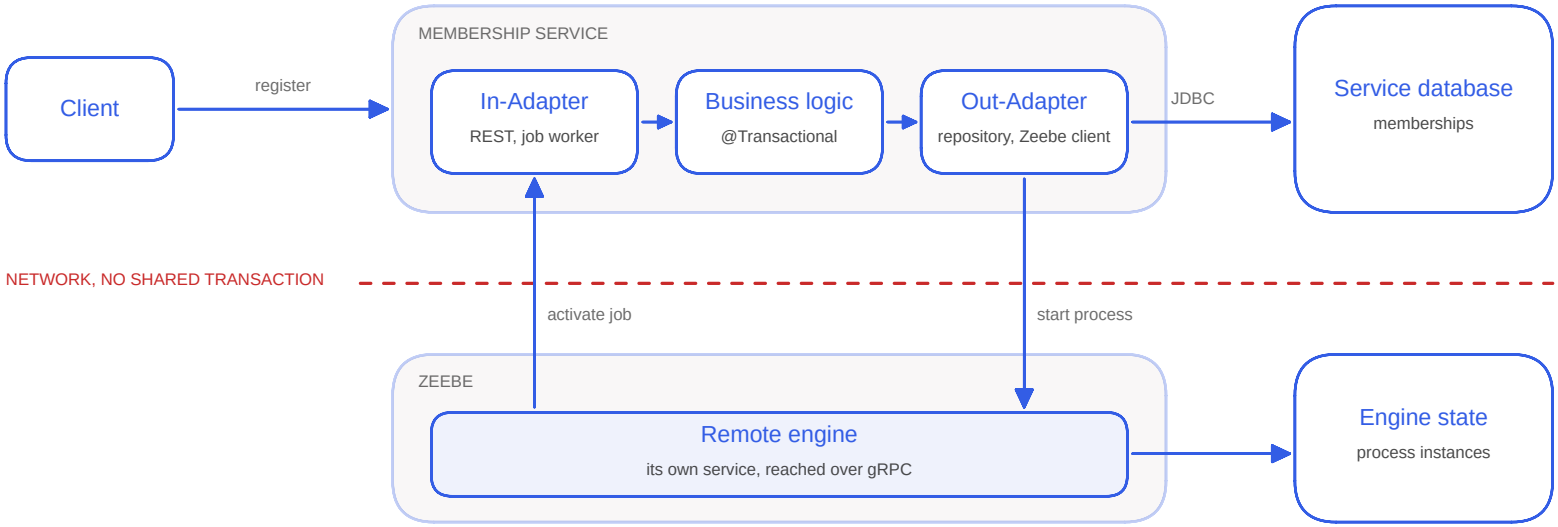
Designed for modern cloud infrastructure (SaaS & On-Prem)

Future-Ready

Modern architecture for future requirements

CHAPTER 2 · WHAT CHANGED

The engine moves out of your service



Two services, two databases, and a network in between

CHAPTER 2 · WHAT STAYED

For developers, it still feels the same

That is why the change is easy to underestimate. The concrete automation follows the same paradigms



Model

Modeling the process with bpmn



Operate

Same tools for operations & modelling



Build

Writing glue-code, testing processes & deploying them

CHAPTER 2 · WHAT STAYED

Only the adapters change

The delegate becomes a worker, the adapter talks to Zeebe. Everything else stays as it was

SendWelcomeMailWorker.kt | MembershipProcessAdapter.kt — KOTLIN

```kotlin
@Component
class SendWelcomeMailWorker(private val useCase: SendWelcomeMailUseCase) {

    private val log = KotlinLogging.logger {}

    @JobWorker(type = "membership.sendWelcomeMail")
    fun sendWelcomeMail(@Variable("membershipId") membershipId: String) {
        log.debug { "Received Zeebe job to send welcome mail: $membershipId" }
        useCase.sendWelcomeMail(MembershipId(UUID.fromString(membershipId)))
    }
}
```

CHAPTER 2 · WHAT STAYED

Only the adapters change

The delegate becomes a worker, the adapter talks to Zeebe. Everything else stays as it was

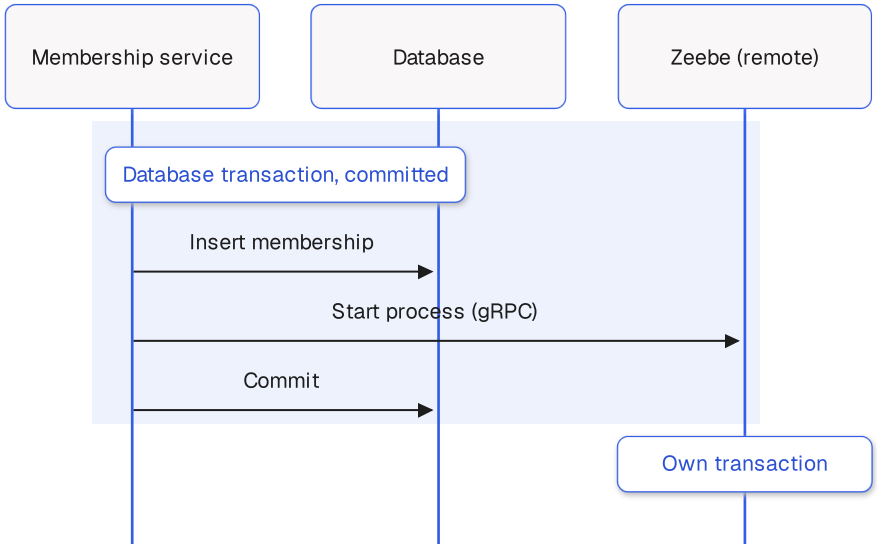
SendWelcomeMailWorker.kt | **MembershipProcessAdapter.kt** | KOTLIN

```kotlin
@Component
class MembershipProcessAdapter(
    private val camundaClient: CamundaClient
) : MembershipProcess {

    override fun submitRegistration(id: MembershipId) {
        val variables = mapOf("membershipId" to id.value.toString())
        camundaClient.newPublishMessageCommand()
            .messageName("miravelo.registrationSubmitted")
            .withoutCorrelationKey()
            .variables(variables)
            .send()
            .join()
    }
}
```

CHAPTER 2 · WHEN IT WORKS

When it works, everyone agrees



The user says

I am a member

The database says

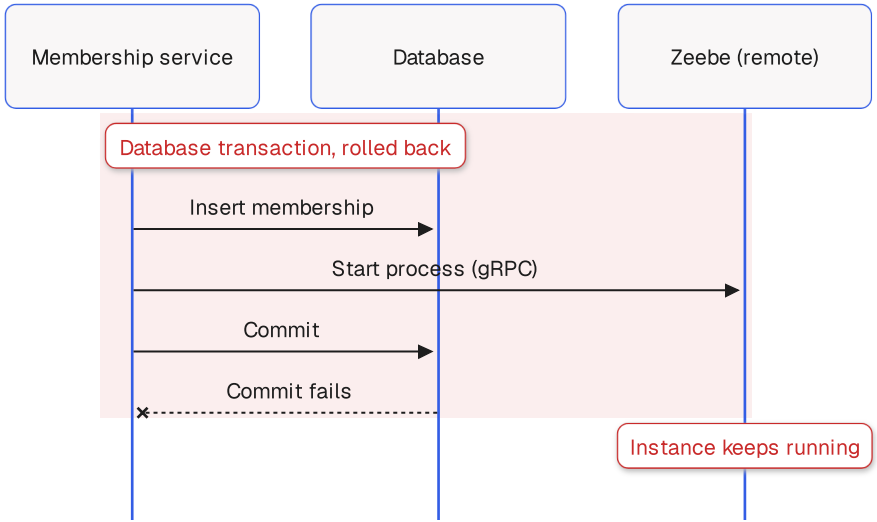
Membership stored

The engine says

Process instance running

CHAPTER 2 · WHEN IT FAILS

When it fails, nobody agrees



The user says

It did not work, I will try again

The database says

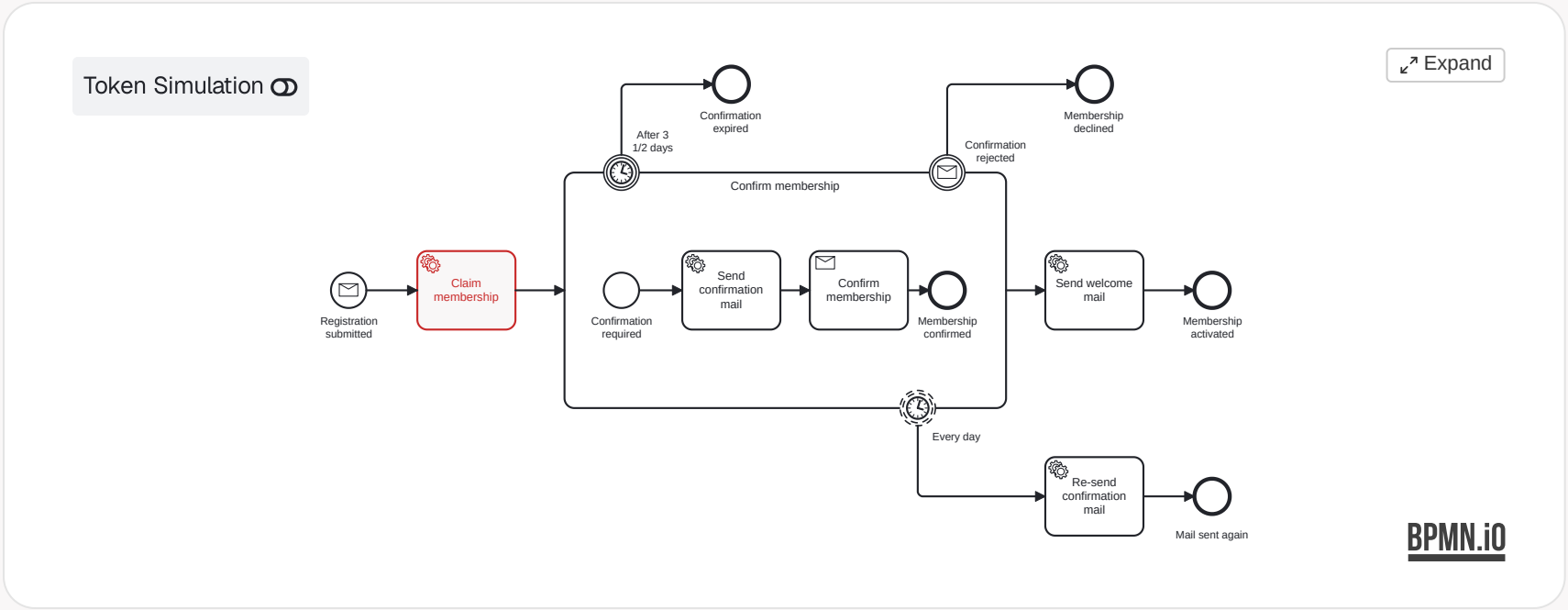
No membership

The engine says

Process instance running

CHAPTER 2 · WHEN IT FAILS

And the process runs into an incident



The worker looks for a membership that was rolled back. This is the ticket from the beginning

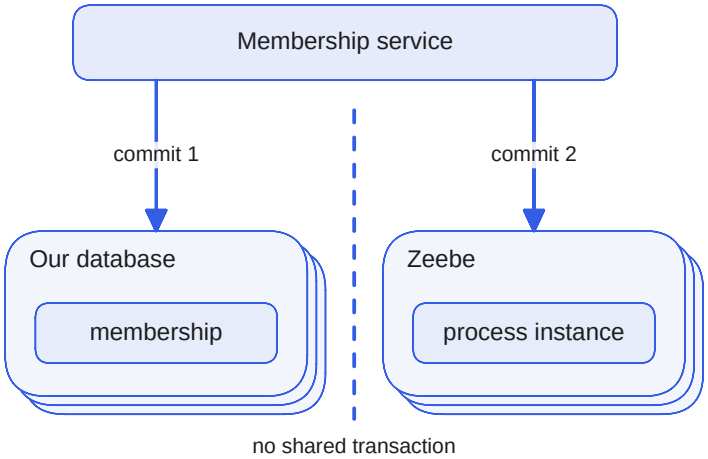
OUR SAFEGUARD IS GONE

Neither all, nor nothing

`@Transactional` still works, but it only covers our own database now

New paradigm: BASE

One service, two systems, two commits

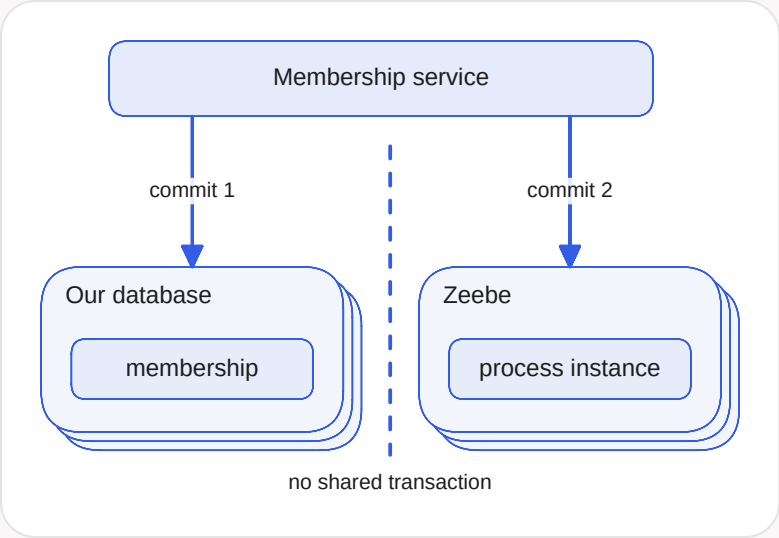


1 **Basically available:** The system keeps answering, even if parts are down

CHAPTER 2 · WHY IT BREAKS

New paradigm: BASE

One service, two systems, two commits

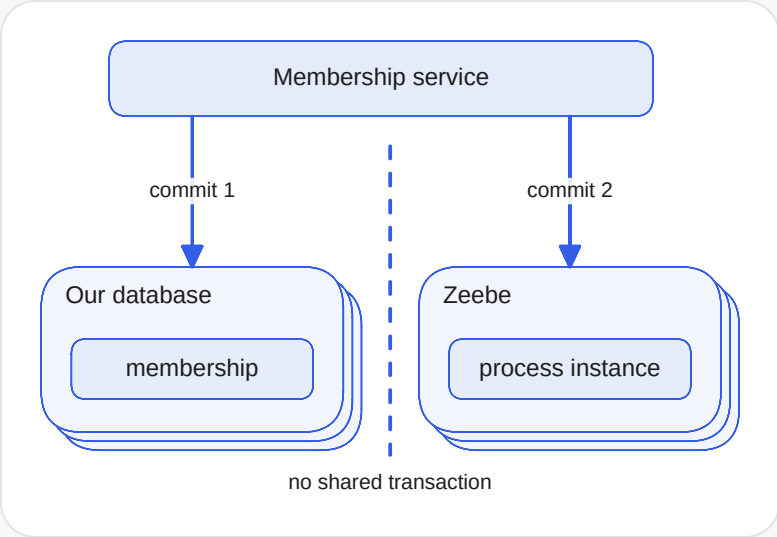


1. **Basically available:** The system keeps answering, even if parts are down
2. **Soft state:** The state still changes while the systems sync

CHAPTER 2 · WHY IT BREAKS

New paradigm: BASE

One service, two systems, two commits



1. **Basically available:** The system keeps answering, even if parts are down
2. **Soft state:** The state still changes while the systems sync
3. **Eventually consistent:** For a while they disagree, in the end they converge

CHAPTER 2 · WHAT WE LOST

Same failure, different ending

The commit fails in both worlds. Only one of them cleans up after itself

Camunda 7

The database says: no membership

The engine says: no process instance

Result: consistent, the user simply tries again

CHAPTER 2 · WHAT WE LOST

Same failure, different ending

The commit fails in both worlds. Only one of them cleans up after itself

Camunda 7

The database says: no membership

The engine says: no process instance

Result: consistent, the user simply tries again

Zeebe

The database says: no membership

The engine says: process instance running

Result: inconsistent, an incident is waiting

CHAPTER 3

The distributed transaction problem

What we just saw has a name

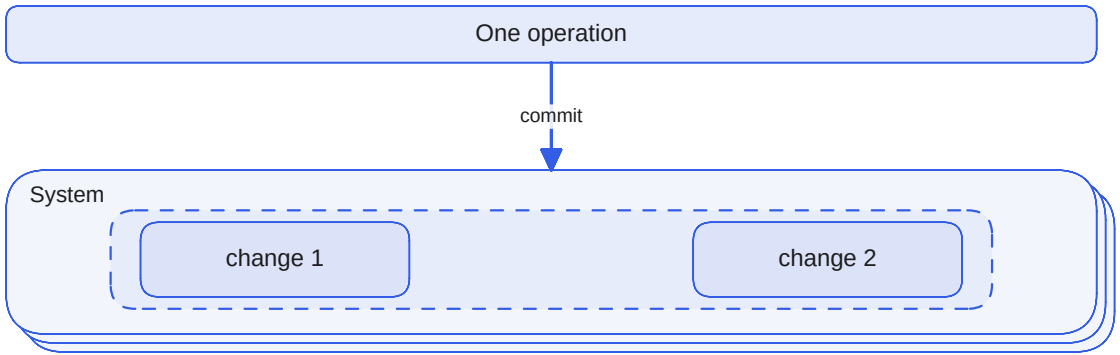
03

CHAPTER 3 · THE DEFINITION

A generic problem, precisely defined

Not tied to any product. One operation, several systems, each commits on its own

One system | Several systems | The problem



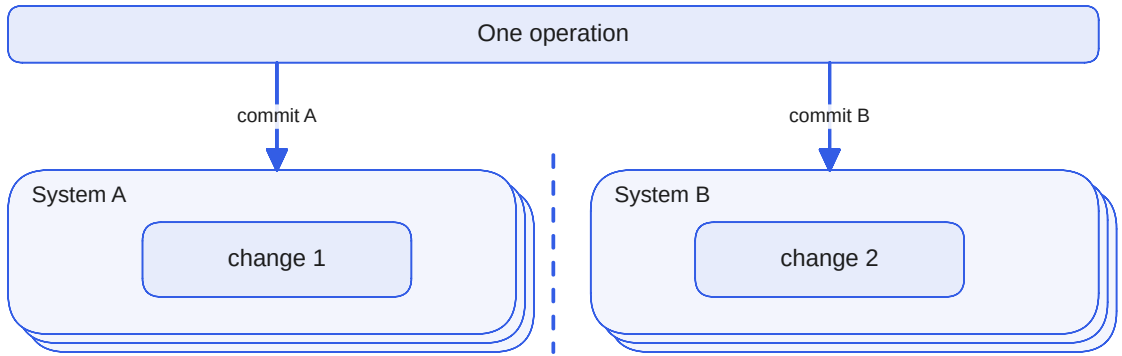
One system, one commit: all or nothing

CHAPTER 3 · THE DEFINITION

A generic problem, precisely defined

Not tied to any product. One operation, several systems, each commits on its own

One system | **Several systems** | The problem



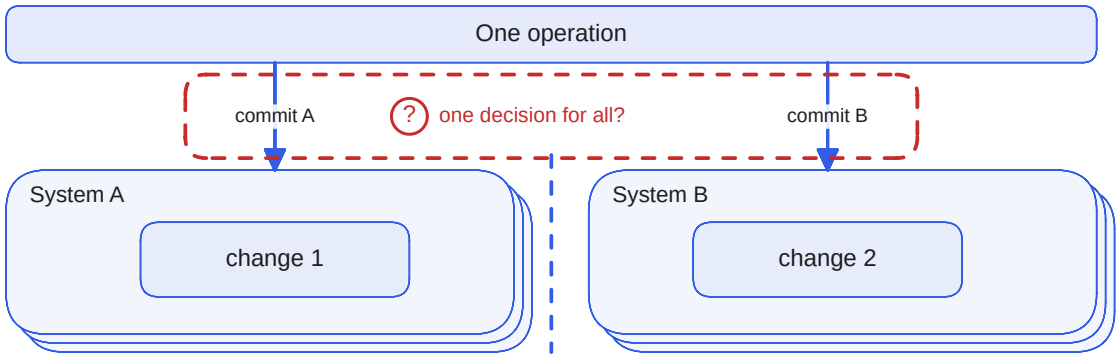
Several systems, separate commits, nothing ties them together

CHAPTER 3 · THE DEFINITION

A generic problem, precisely defined

Not tied to any product. One operation, several systems, each commits on its own

One system | Several systems | **The problem**



Distributed transaction problem: reach one all-or-nothing outcome across systems that each commit alone

CHAPTER 3 · THE THEORY

Four outcomes, two of them hurt

Two systems, each can succeed or fail on its own. That leaves four combinations

A succeeds, B succeeds

Consistent. The happy path

A fails, B fails

Consistent. Nothing happened, try again

CHAPTER 3 · THE THEORY

Four outcomes, two of them hurt

Two systems, each can succeed or fail on its own. That leaves four combinations

A succeeds, B succeeds

Consistent. The happy path

A fails, B fails

Consistent. Nothing happened, try again

A succeeds, B fails

Inconsistent. Later: the service is ahead

CHAPTER 3 · THE THEORY

Four outcomes, two of them hurt

Two systems, each can succeed or fail on its own. That leaves four combinations

A succeeds, B succeeds

Consistent. The happy path

A fails, B fails

Consistent. Nothing happened, try again

A succeeds, B fails

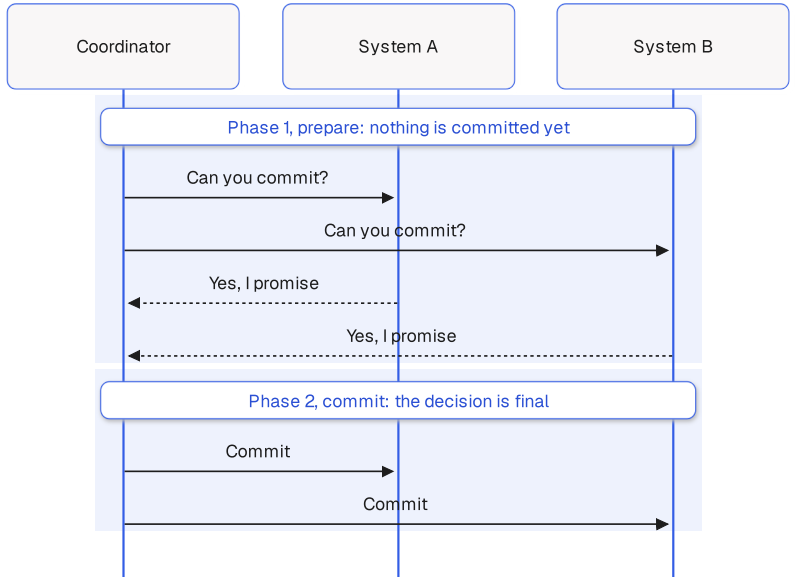
Inconsistent. Later: the service is ahead

A fails, B succeeds

Inconsistent. Later: the engine is ahead

CHAPTER 3 · THE THEORY

The textbook fix: two-phase commit



Phase 1: ask

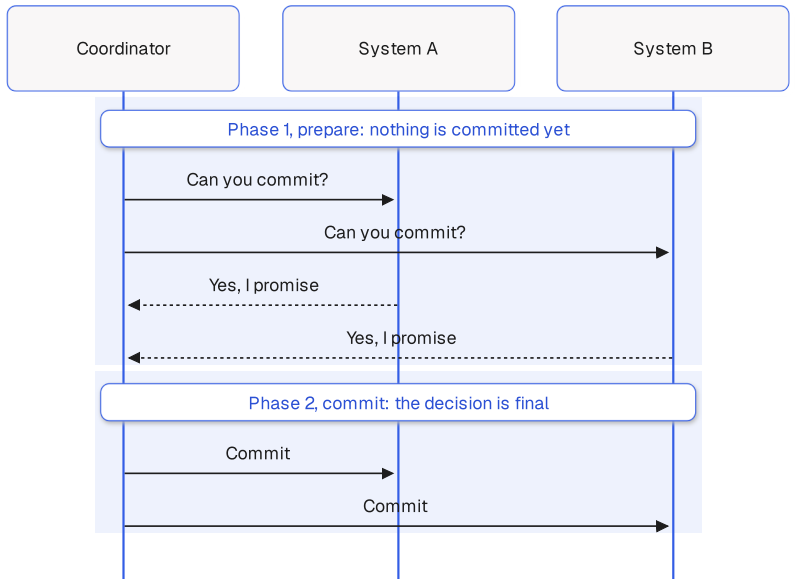
Everyone prepares, keeps its locks and votes

Phase 2: tell

All voted yes, so the decision is final

CHAPTER 3 · THE THEORY

The textbook fix: two-phase commit



Phase 1: ask

Everyone prepares, keeps its locks and votes

Phase 2: tell

All voted yes, so the decision is final

And if a commit fails now?

A yes is a durable promise. Whoever crashes still has to commit after recovery

CHAPTER 3 · THE THEORY

Why it is not our way out

The guarantee is real, but it is bought with availability

Everyone must speak it

Zeebe, REST APIs and mail servers do not support the protocol

Everyone must wait

Locks are held until the last one has committed

Everyone must be up

One crash blocks all the others

CHAPTER 3 · THE THEORY

Why it is not our way out

The guarantee is real, but it is bought with availability

Everyone must speak it

Zeebe, REST APIs and mail servers do not support the protocol

Everyone must wait

Locks are held until the last one has committed

Everyone must be up

One crash blocks all the others

Two-phase commit cannot solve the problem between Zeebe and our database.

So we have to look closely at where exactly it hits us

FROM THEORY TO PRACTICE

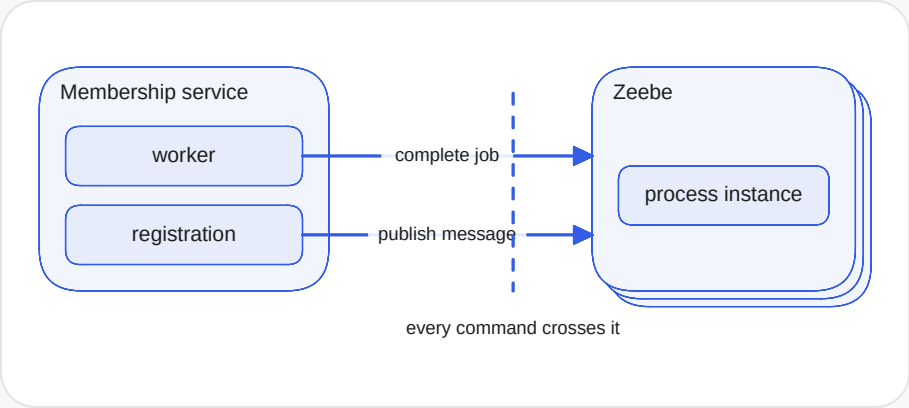
Where does it bite in Zeebe?

Every command we send to the engine crosses a system boundary

CHAPTER 3 · IN ZEEBE

Two triggers, three outcomes

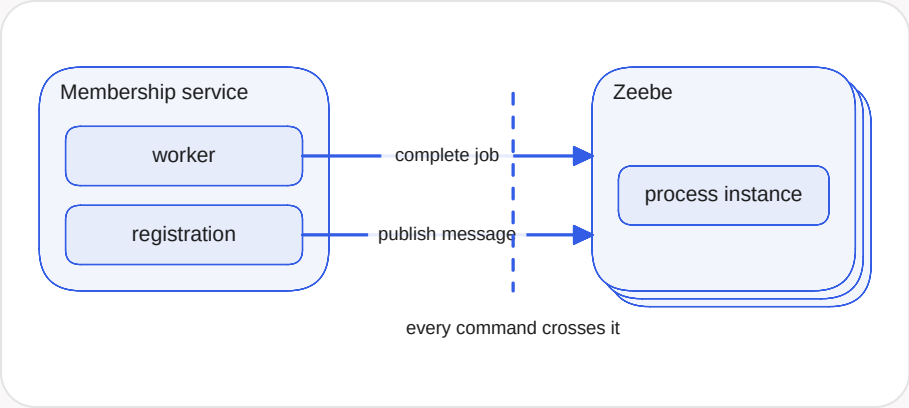
Two kinds of commands can leave the system inconsistent



CHAPTER 3 · IN ZEEBE

Two triggers, three outcomes

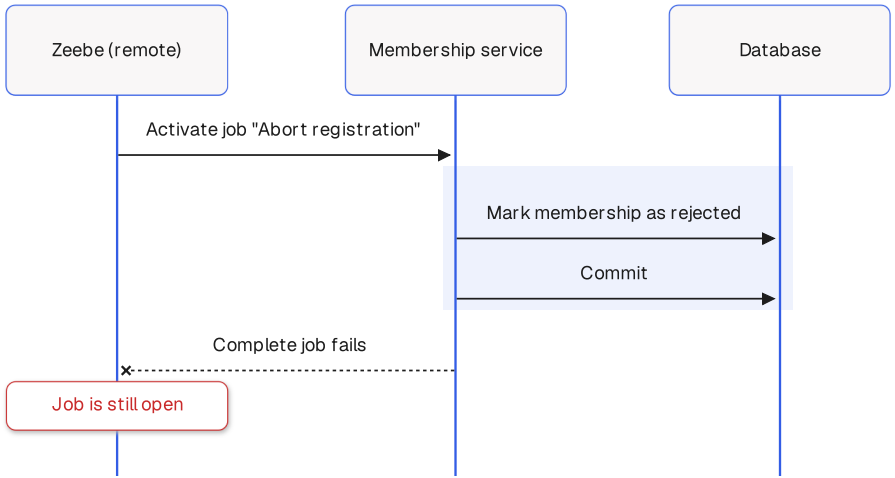
Two kinds of commands can leave the system inconsistent



1. **Service ahead:** Our data is saved, the engine knows nothing
2. **Engine ahead:** The engine moved on, our data is missing
3. **Incompatible:** Both moved on, into states that no longer fit

CHAPTER 3 · IN ZEEBE · COMPLETING A JOB

Our data is saved, but the job stays open



The database says

Membership rejected

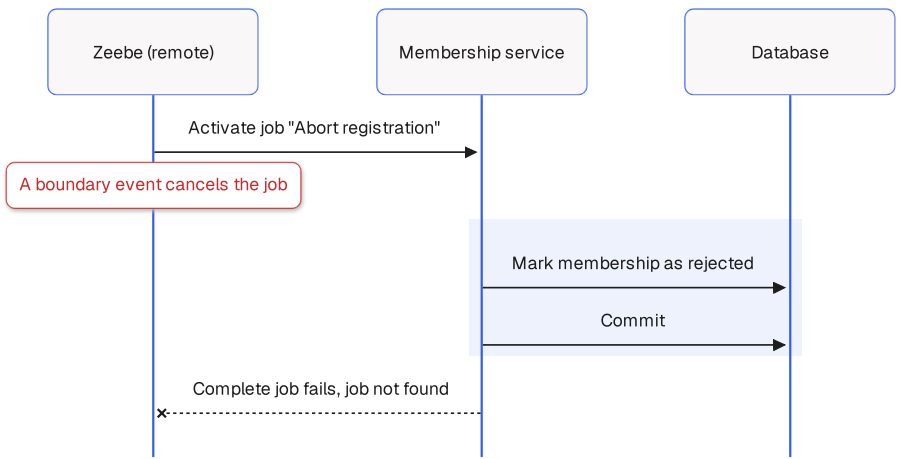
The engine says

Job is still open

Service ahead

Often temporary. A retry fixes it, but may cause side effects

Sometimes no retry can fix it



The database says

Membership rejected

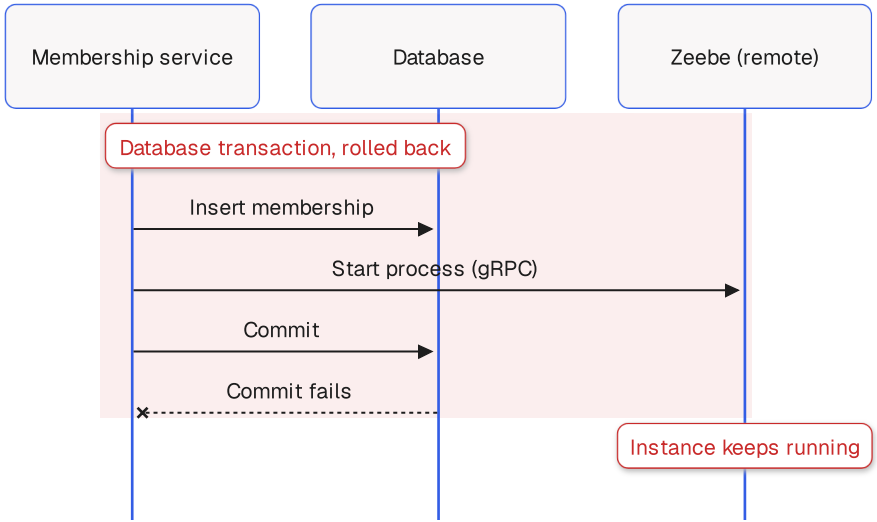
The engine says

Job cancelled by a boundary event

Incompatible

Permanent. No retry will ever succeed

The engine gets the message, our data is lost



The database says

No membership

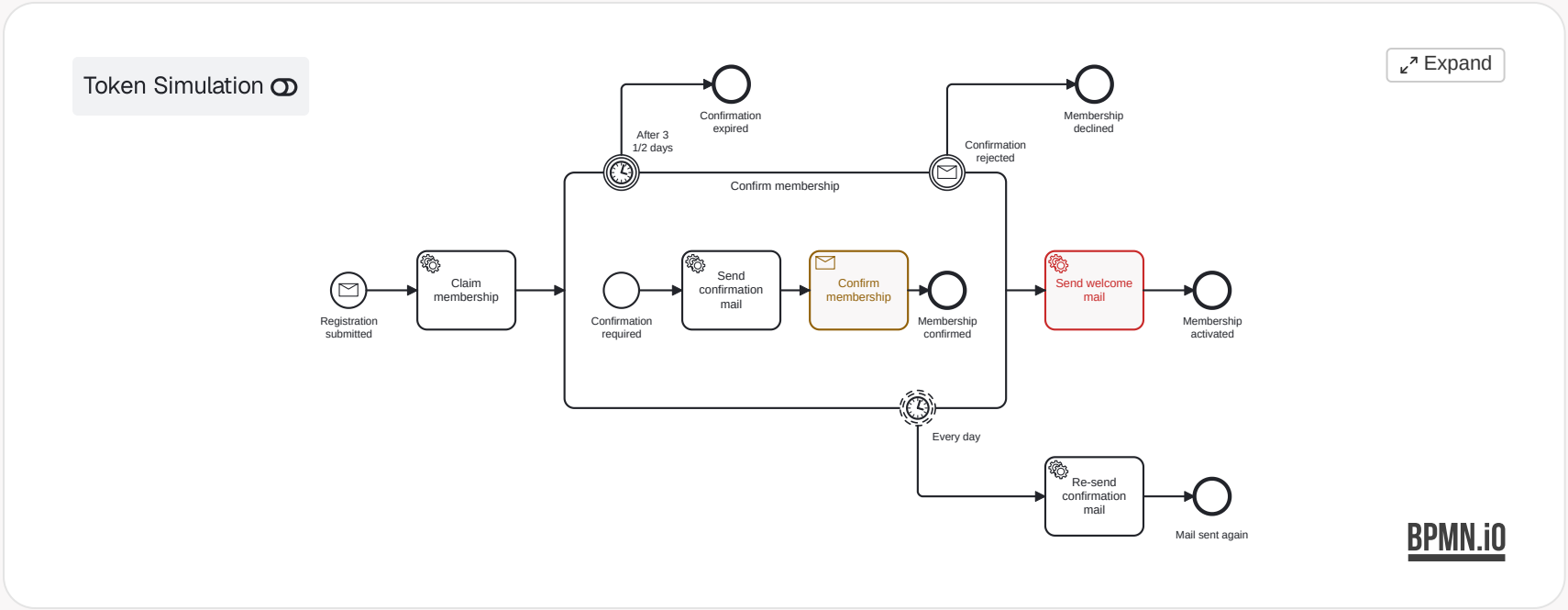
The engine says

Process instance running

Engine ahead

Workers fail, because the data they need was never saved

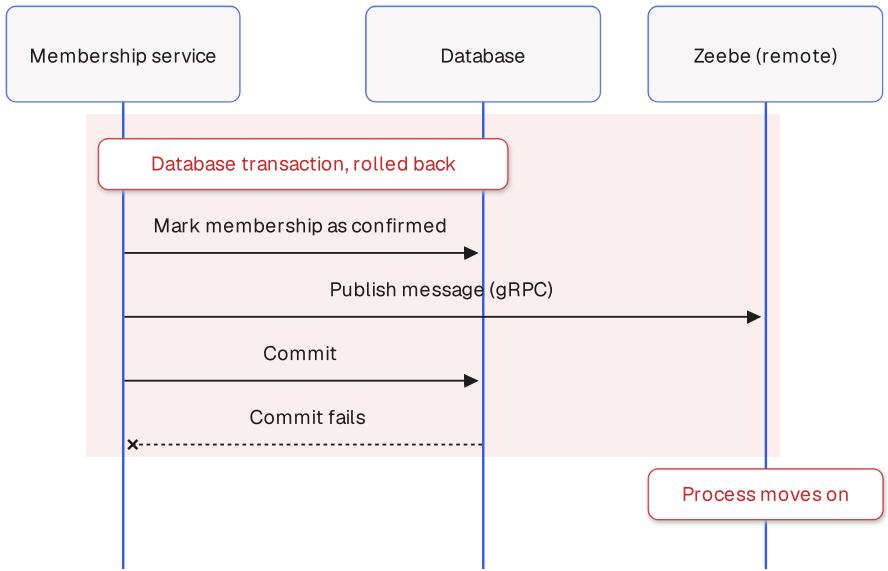
It gets worse in the middle of a process



Take the moment the user confirms the membership

CHAPTER 3 · IN ZEEBE · PUBLISHING A MESSAGE

The engine moves on with data we never saved



The database says

Membership not confirmed

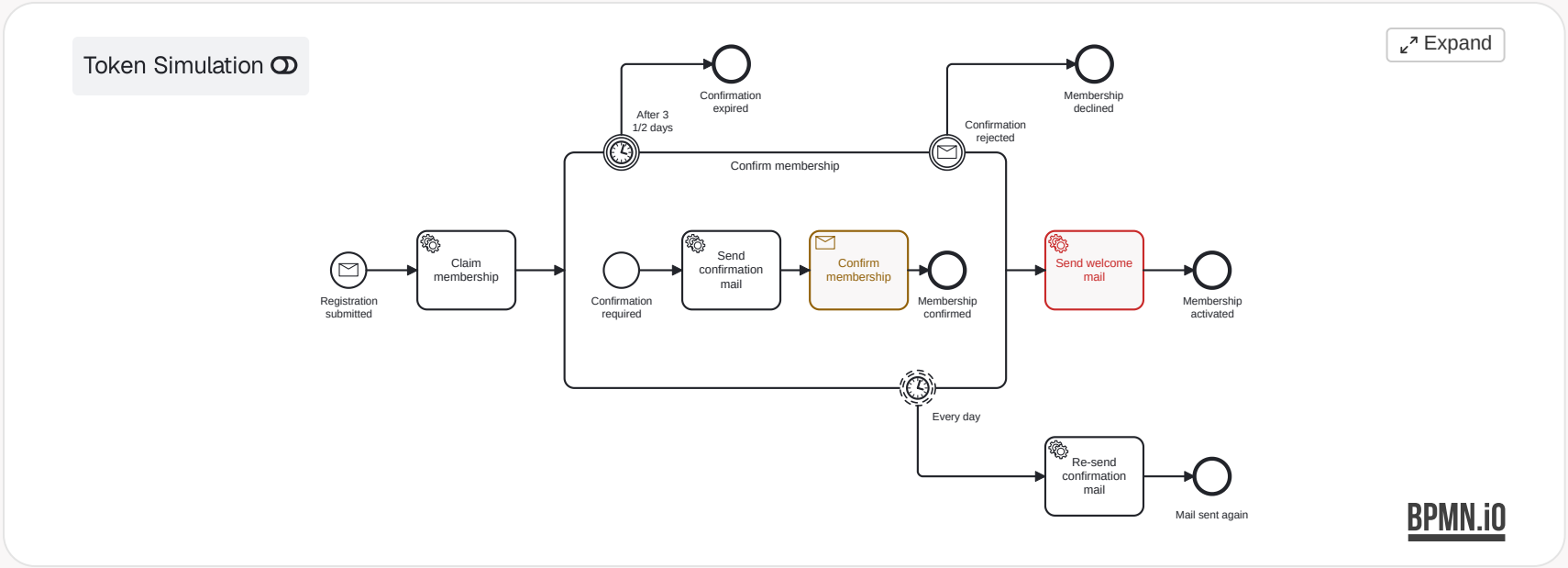
The engine says

Correlated, next task started

Engine ahead, again

This time the next task finds its data, just not in the right state

And the next task may even succeed



Incompatible: the welcome mail goes out for a membership that was never confirmed. In other processes such a task could corrupt data

WHAT WE LEARN FROM THIS

It comes down to timing and delivery

The engine hears of our data before it is saved, or never hears that a job is done

CHAPTER 4

Exploring solutions

Three attempts to narrow the gap

04

CHAPTER 4 · THE YARDSTICK

What a solution has to deliver

We judge every attempt by the same three questions. They come straight out of chapter 3



Timing

Does the engine only hear of data that is safely committed?



Delivery

Is the message guaranteed to arrive, even after a crash?



Price

What do we pay for it in code, infrastructure and new problems?

CHAPTER 4 · ATTEMPT 1 OF 3

Retry

If some part fails, just run the whole operation again

4.1

CHAPTER 4 · ATTEMPT 1 · RETRY

Why a retry feels right

It is the standard answer to failures in one system. It rests on three quiet assumptions



The error is temporary

A network blip or a busy server. Wait a moment and it works



Repeating is harmless

Doing the same thing twice leaves the same result as doing it once



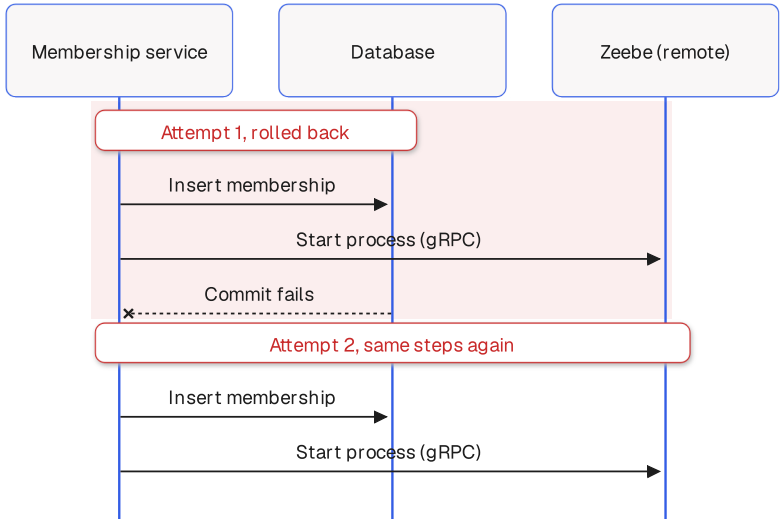
Nothing happened yet

A failed attempt left no traces anywhere

Across two systems, none of the three is guaranteed

CHAPTER 4 · ATTEMPT 1 · RETRY

A retry repeats the engine call too



Errors are often permanent

A constraint violation fails on every retry

Retries may start a duplicate

Zeebe does not know the first call belonged to a failed attempt

The order is still wrong

The engine still hears of uncommitted data

CHAPTER 4 · ATTEMPT 1 · RETRY

Verdict: more problems than it solves

The first thing that comes to mind, and rarely a help. What we take from it: tell Zeebe only after a successful commit

Timing

Unsolved. The engine still runs ahead of our data

CHAPTER 4 · ATTEMPT 1 · RETRY

Verdict: more problems than it solves

The first thing that comes to mind, and rarely a help. What we take from it: tell Zeebe only after a successful commit



Timing

Unsolved. The engine still runs ahead of our data



Delivery

No guarantee, but duplicates on top

CHAPTER 4 · ATTEMPT 1 · RETRY

Verdict: more problems than it solves

The first thing that comes to mind, and rarely a help. What we take from it: tell Zeebe only after a successful commit



Timing

Unsolved. The engine still runs ahead of our data



Delivery

No guarantee, but duplicates on top



Price

Cheap to build, expensive to operate

CHAPTER 4 · ATTEMPT 2 OF 3

After transaction

Call the engine only once our own data is safe

4.2

CHAPTER 4 · ATTEMPT 2 · AFTER TRANSACTION

Why the order matters

Retry failed because the engine heard of data we never saved. So we turn the order of the two steps around



Commit first

Our own data is safe before anyone else hears of it



Engine second

The call is no longer part of the transaction. It follows a successful commit



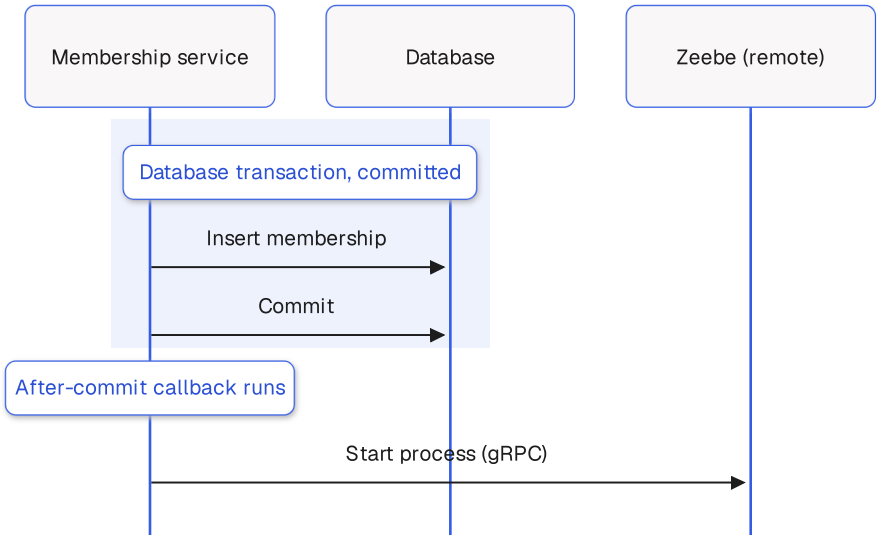
The hook exists

Spring can run code right after a commit. We only have to register it

A failed commit now simply means: nothing happened

CHAPTER 4 · ATTEMPT 2 · AFTER TRANSACTION

The engine only hears of committed data



The database says

Membership stored

The engine says

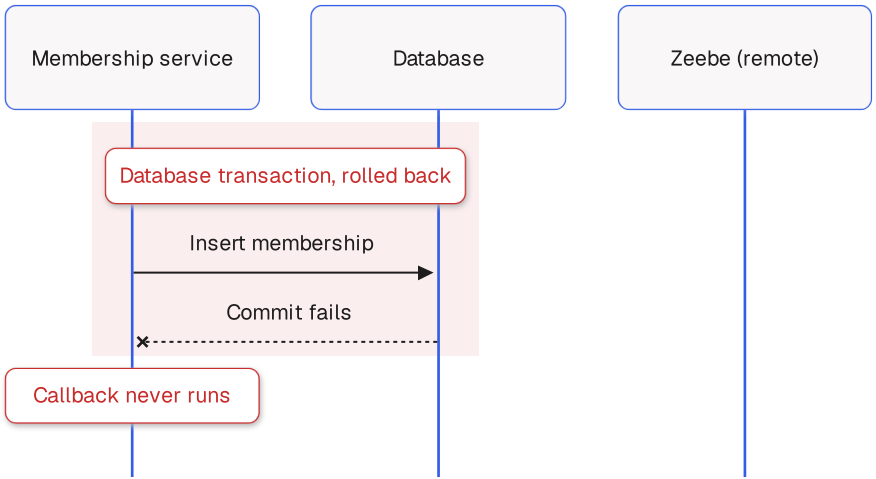
Process instance running

Timing solved

Zeebe is told after the commit, never before

CHAPTER 4 · ATTEMPT 2 · AFTER TRANSACTION

A failing commit never reaches Zeebe



The database says

No membership

The engine says

Nothing, and that is correct

Consistent

The user simply tries again, as in the Camunda 7 days

CHAPTER 4 · ATTEMPT 2 · AFTER TRANSACTION

Spring hooks into the commit

After commit | Before commit

KOTLIN

```kotlin
fun executeAfterCommit(processEngineCall: () -> Unit) {
    log.debug { "Registering process engine call after commit" }
    registerEngineCallAfterCommit(processEngineCall)
}

private fun registerEngineCallAfterCommit(
    processEngineCall: () -> Unit
) = TransactionSynchronizationManager.registerSynchronization(
    object : TransactionSynchronization {
        override fun afterCommit() = try {
            processEngineCall()
        } catch (e: Exception) {
            log.error(e) { "Manual action required. Failed to execute engine call" }
            throw e
        }
    }
)
```

The engine call is registered and only runs after the commit

CHAPTER 4 · ATTEMPT 2 · AFTER TRANSACTION

Spring hooks into the commit

After commit | **Before commit** | KOTLIN

```kotlin
override fun beforeCommit(readOnly: Boolean) {
    val topology = zeebeClient.newTopologyRequest().send().join()
    val healthy = checkBrokerHealth(topology)
    if (!healthy) {
        throw IllegalStateException("No healthy broker found")
    }
}

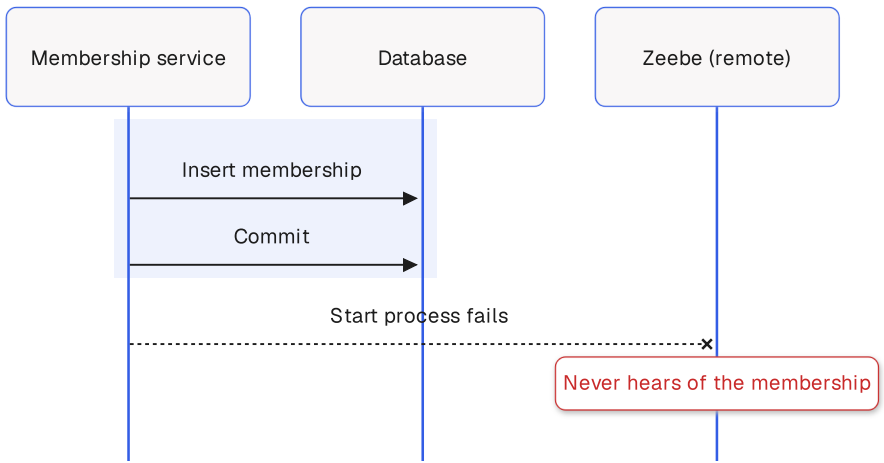
private fun checkBrokerHealth(topology: Topology): Boolean {
    return topology.brokers.any { brokerIsHealthy(it) }
}

private fun brokerIsHealthy(broker: BrokerInfo): Boolean {
    return broker.partitions.any {
        it.health == PartitionBrokerHealth.HEALTHY
    }
}
```

A health check before the commit lowers the risk of a failing call

CHAPTER 4 · ATTEMPT 2 · AFTER TRANSACTION

But the call itself can still fail



The database says

Membership stored

The engine says

Nothing, no process instance

Service ahead

No retry in the pattern. The message is most likely lost

CHAPTER 4 · ATTEMPT 2 · AFTER TRANSACTION

Verdict: easy, but no guarantee

Fine for a proof of concept and uncritical flows. Not for anything that must not get lost

Timing

Solved. The engine never runs ahead of our data

CHAPTER 4 · ATTEMPT 2 · AFTER TRANSACTION

Verdict: easy, but no guarantee

Fine for a proof of concept and uncritical flows. Not for anything that must not get lost

Timing

Solved. The engine never runs ahead of our data

Delivery

Not guaranteed. A failed call or a crash loses the message

CHAPTER 4 · ATTEMPT 2 · AFTER TRANSACTION

Verdict: easy, but no guarantee

Fine for a proof of concept and uncritical flows. Not for anything that must not get lost

Timing

Solved. The engine never runs ahead of our data

Delivery

Not guaranteed. A failed call or a crash loses the message

Price

Low. One wrapper, optionally a broker health check

CHAPTER 4 · ATTEMPT 3 OF 3

Outbox

Store the message next to the data, deliver it later

4.3

CHAPTER 4 · ATTEMPT 3 · OUTBOX

Why a table solves it

After transaction lost the message because it only lived in memory. So we write it down

Stored together

The message goes into our database, in the same transaction as the business data

Survives a crash

What is committed is still there after a restart. Nothing depends on memory any more

Delivered later

A separate processor sends it to Zeebe and keeps trying until it worked

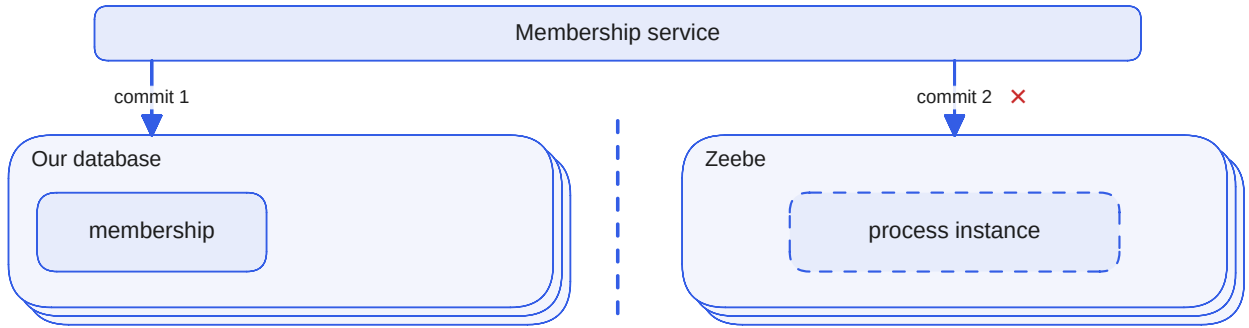
Known as the transactional outbox pattern

CHAPTER 4 · ATTEMPT 3 · OUTBOX

Two systems become one again

One commit into our own database. Membership and message are stored atomically

Without outbox | With outbox · two phases



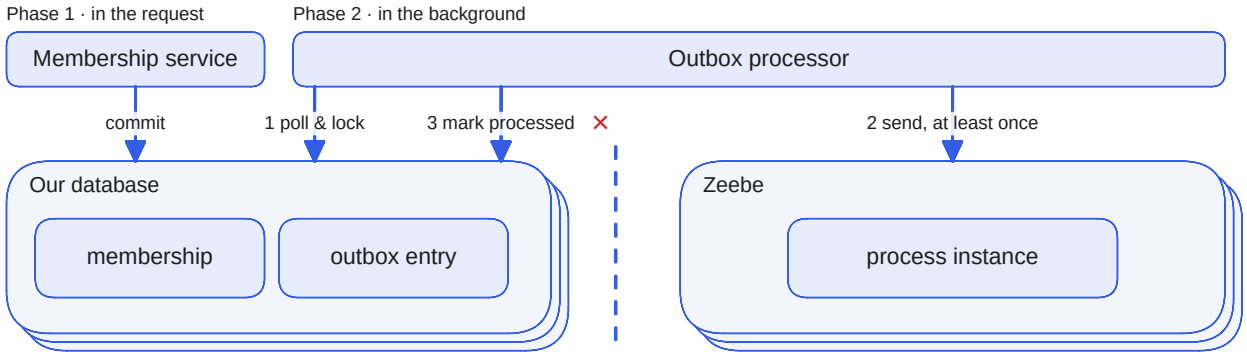
Membership saved, no process started

CHAPTER 4 · ATTEMPT 3 · OUTBOX

Two systems become one again

One commit into our own database. Membership and message are stored atomically

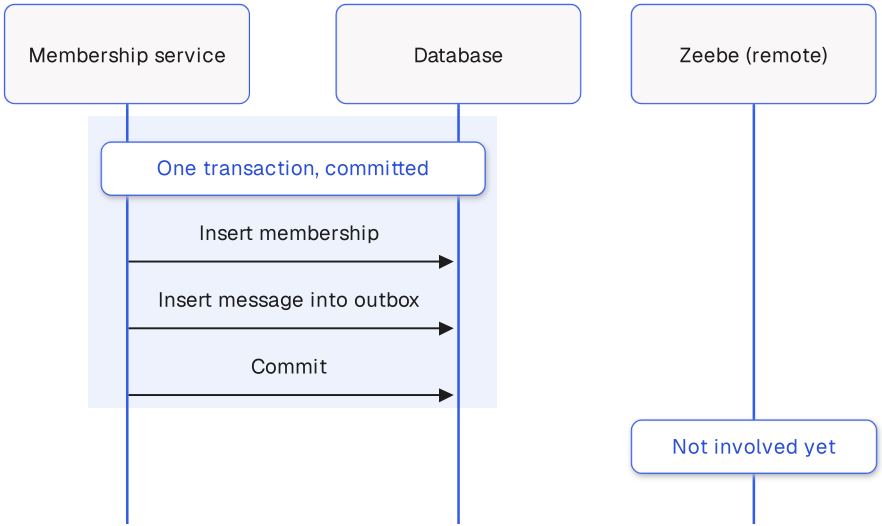
Without outbox | With outbox · two phases



One commit in the request, delivery in the background: a crash before 3 repeats 2

CHAPTER 4 · ATTEMPT 3 · OUTBOX

Phase 1: data and message, all or nothing



The database says

Membership and message stored

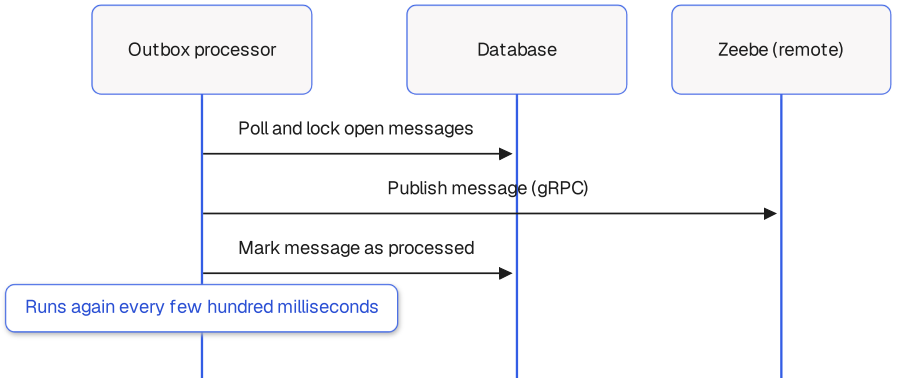
The engine says

Nothing yet

Almost Camunda 7

Instead of starting the process we store the intent to start it

Phase 2: a processor delivers until it worked



The database says

Message processed

The engine says

Process instance running

Delivery guaranteed

A failed call stays in the outbox and is retried

CHAPTER 4 · ATTEMPT 3 · OUTBOX

The price: three trade-offs

More infrastructure

Delivery decisions

At least once

- **Sending is no longer one call to Zeebe**
- **To build:** an extra table, a scheduler and the code around them
- **To keep:** code we run and maintain

CHAPTER 4 · ATTEMPT 3 · OUTBOX

The price: three trade-offs

More infrastructure

Delivery decisions

At least once

- **One by one:** safe, but slow, and one stuck message jams all others
- **Several at once:** faster, but only safe in order, by creation time or per process instance
- **No one-size-fits-all:** decide by your architectural characteristics

CHAPTER 4 · ATTEMPT 3 · OUTBOX

The price: three trade-offs

More infrastructure

Delivery decisions

At least once

- **Sent, but not marked as sent:** the next run sends the message again
- **So not just one goes out:** Zeebe may receive a message twice or more
- **Our system must be able to handle duplicates**

CHAPTER 4 · ATTEMPT 3 · OUTBOX

An idea: one lane per key, sent right away

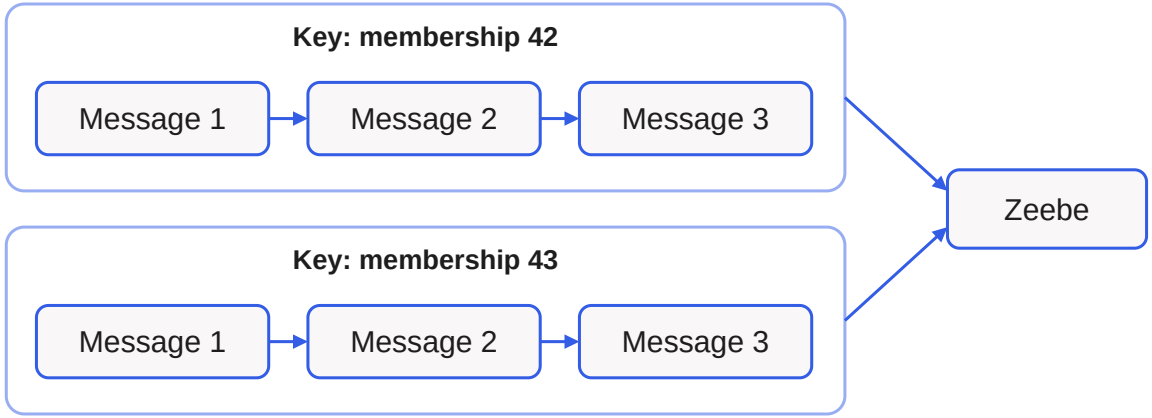
One lane per key

One stuck key

Fast path, safety net

Each message carries a key, its process instance

Same key, same lane, strict order



CHAPTER 4 · ATTEMPT 3 · OUTBOX

An idea: one lane per key, sent right away

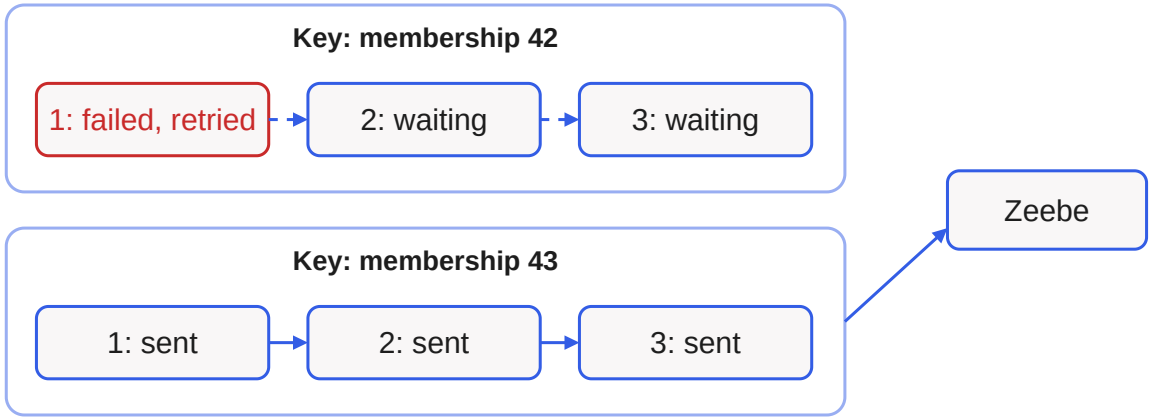
One lane per key

One stuck key

Fast path, safety net

A failing message stops only its own lane

All other keys keep flowing



CHAPTER 4 · ATTEMPT 3 · OUTBOX

An idea: one lane per key, sent right away

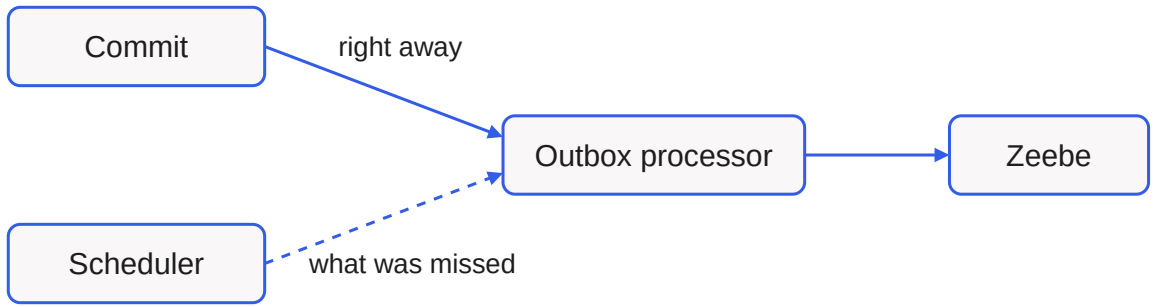
One lane per key

One stuck key

Fast path, safety net

Sent right after the commit

A scheduler catches the rest. Still at least once



CHAPTER 4 · ATTEMPT 3 · OUTBOX

Verdict: the way to go, with duplicates

The only attempt that never loses a message

Timing

Solved. Delivery is separated from the business transaction

CHAPTER 4 · ATTEMPT 3 · OUTBOX

Verdict: the way to go, with duplicates

The only attempt that never loses a message

Timing

Solved. Delivery is separated from the business transaction

Delivery

Guaranteed, at least once

CHAPTER 4 · ATTEMPT 3 · OUTBOX

Verdict: the way to go, with duplicates

The only attempt that never loses a message

Timing

Solved. Delivery is separated from the business transaction

Delivery

Guaranteed, at least once

Price

A table, a processor, an ordering strategy, and idempotent receivers

THREE ATTEMPTS, ONE WINNER

The outbox wins, but the gap stays

No attempt closes it. Each one trades one risk for another

CHAPTER 5

Mastering idempotency

The price of the winner, paid in three places

05

CHAPTER 5 · THE DEFINITION

Repeating must not hurt

An operation is idempotent if one of these holds

Either

Repeating it always produces the same result

Or

Repeating it causes no harm

Three places to build it in

In your code

For workers: a job that comes twice

With Zeebe

For messages: a copy that arrives twice

In the model

For both: fewer places where it hurts

CHAPTER 5 · 1/3 · IN YOUR CODE

Two ways to handle a repeated job

Run it again

Detect and ignore it

Safe by nature

The action has no side effects. Running it twice leaves the same state

MEMBERSHIP

id	42
status	REJECTED

CHAPTER 5 · 1/3 · IN YOUR CODE

Two ways to handle a repeated job

Run it again

Detect and ignore it

Safe by design

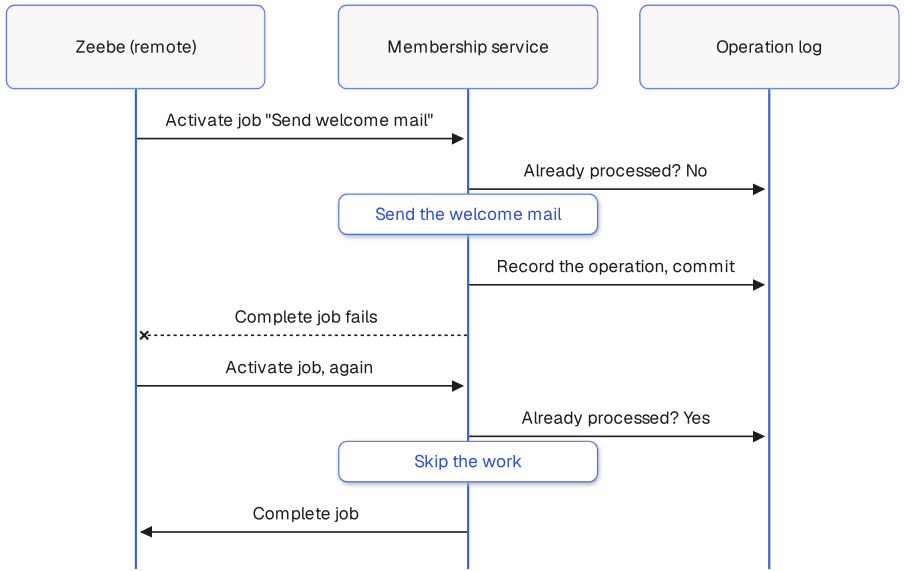
We add a flag to our data model and check it before we act

MEMBERSHIP

id	42
status	CONFIRMED
welcome_mail_sent	true

CHAPTER 5 · 1/3 · IN YOUR CODE

An operation log: the outbox turned around



Check, execute, record

All three in one transaction

The outbox turned around

We log what came in, not what leaves

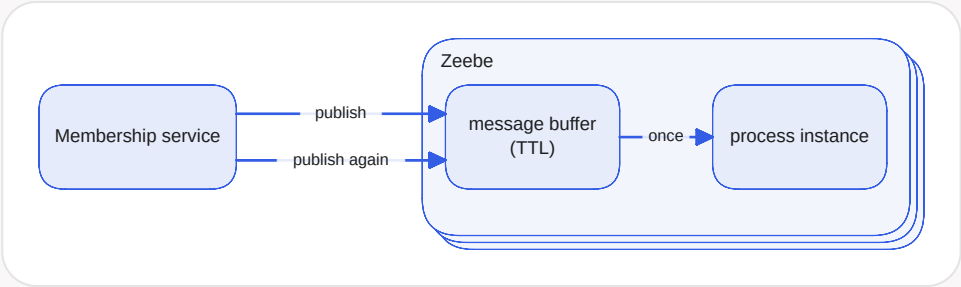
Better, not perfect

A third system can still see it twice

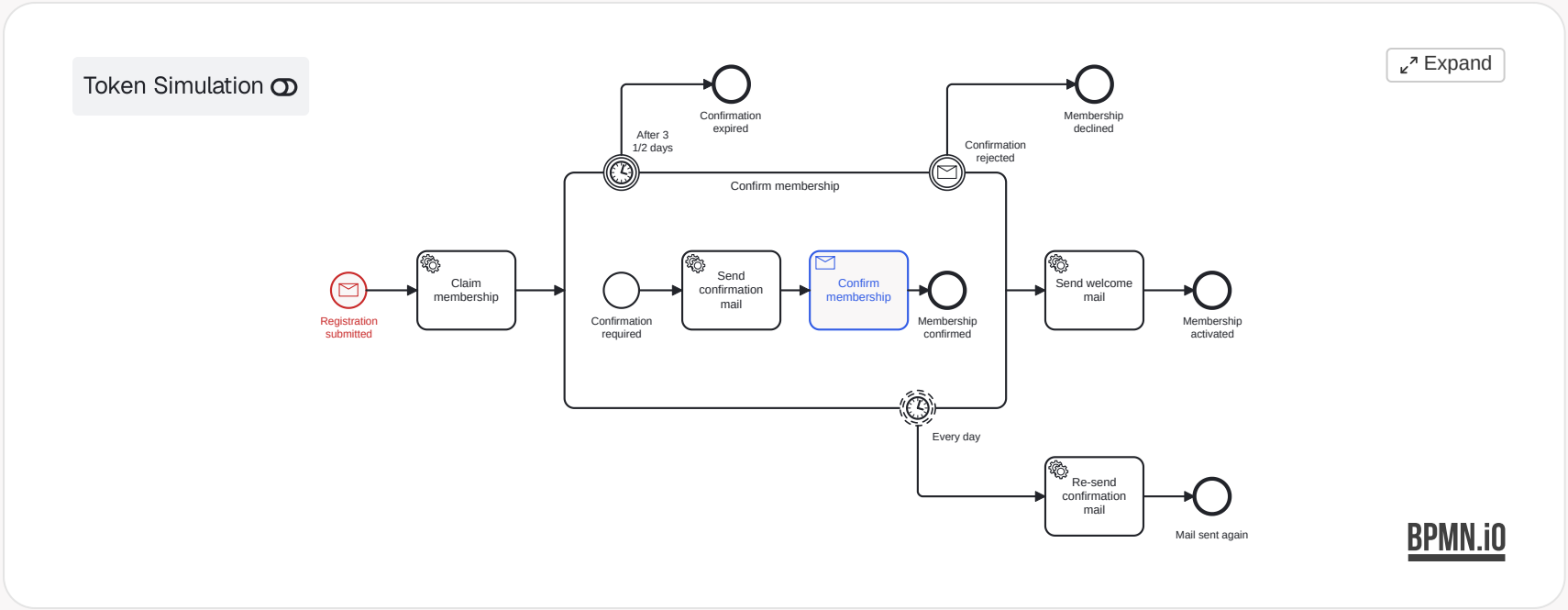
Are messages idempotent already?

You might think so. Send a message twice, and the process moves on only once

1. **Always accepted:** Whether or not an instance waits for it
2. **Buffered:** Kept in the broker for its time-to-live
3. **Correlated once:** Per process, across all its versions



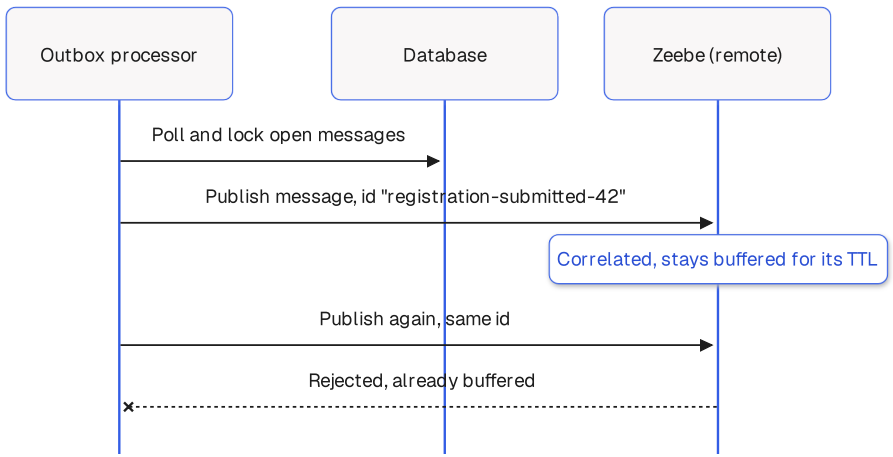
Harmless at the receive task, not at the start



Receive task: copies find nobody waiting. Start event: no key, an instance per copy. Key: one active instance

CHAPTER 5 · 2/3 · WITH ZEEBE

That is why Zeebe added the messageId



An idempotency key

Sent along with every message

Every later copy

Rejected: same name, key and id

The catch

Only while the first is buffered

One line sets it

MembershipProcessAdapter.kt KOTLIN

```kotlin
override fun submitRegistration(id: MembershipId) {
    camundaClient.newPublishMessageCommand()
        .messageName("miravelo.registrationSubmitted")
        .correlationKey(id.value.toString())
        .messageId("registration-submitted-${id.value}")
        .timeToLive(Duration.ofHours(1))
        .variables(variables)
        .send()
        .join()
}
```

With the Java client, in Spring too. We derive the id from the business key, so every retry sends the same one

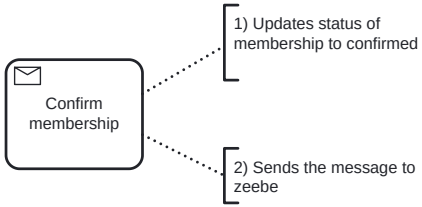
CHAPTER 5 · 3/3 · IN THE PROCESS MODEL

Split tasks that do two things

The rule: one task writes to one system. Reading from others is fine

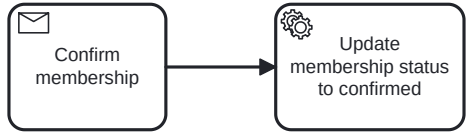
Before

One task: status **and** message

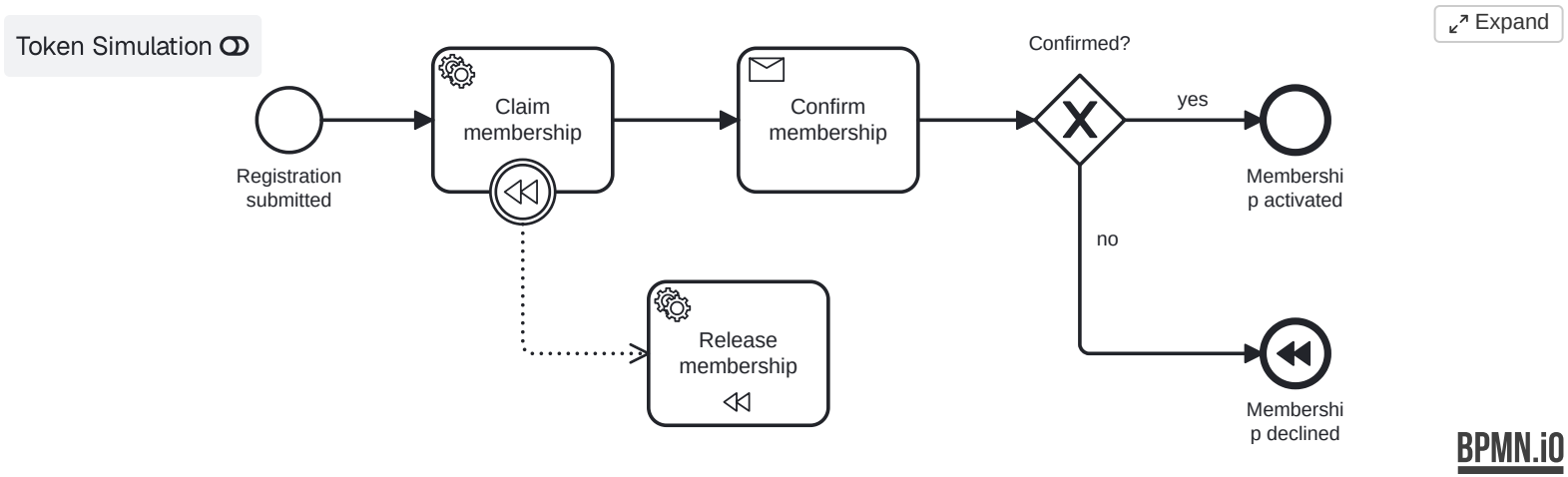


After

One task each: message, then status



An idea: could an undo step help?



Across systems there is no rollback. But BPMN can model the undo, as a compensation

IN YOUR CODE, WITH ZEEBE, IN THE MODEL

None of the three closes the gap

Together they make it small enough to handle. And every sender needs an idempotent receiver, not just the outbox

CHAPTER 6

Our Learnings

What to take home, and what to do with it

06

CHAPTER 6 · LEARNING 1

The world has changed

From an ACID world of monoliths into a BASE world of distributed systems

Camunda 7

One database

One transaction

Result: all or nothing, guaranteed

Zeebe

Many components, each with its own state

No shared transaction

Result: distributed transaction problems

CHAPTER 6 · LEARNING 2

It is not a Zeebe problem

Whenever one operation writes to several systems, you have it. The pattern is always the same

Database + Zeebe

What we looked at today

Database + Kafka

The event is out, the commit fails

Database + REST API

The call went through, the commit fails

The good news: the same patterns apply everywhere. And so do their trade-offs

CHAPTER 6 · LEARNING 2

The whole team has to know it

Not everyone has to implement the patterns. But everyone should know what they mean for their area



Developers

Implement the patterns



Product and UX

Design for "not yet consistent"



QA

Test the failure scenarios



Operations

Monitor differently

CHAPTER 6 · LEARNING 2

The whole team has to know it

Not everyone has to implement the patterns. But everyone should know what they mean for their area



Developers

Implement the patterns



Product and UX

Design for "not yet consistent"



QA

Test the failure scenarios



Operations

Monitor differently

My tip: a brown bag session. One hour, the problem, the solutions, and what it means for your system

CHAPTER 6 · LEARNING 3

Design your systems for it

1: Detect

2: Prioritize

3: Solve

- **Where do I leave my system?** External APIs, Zeebe processes, messages, webhooks
- **For each of these calls:** where are my transaction limits?
- **And:** what happens in case of an error?

CHAPTER 6 · LEARNING 3

Design your systems for it

1: Detect

2: Prioritize

3: Solve

Not everything has to be perfect

But you need to know where the critical issues are

Red: must be resolved
Important processes stop, downtime is expensive

Yellow: should be resolved
Noticeable, but no showstopper

Green: could be resolved
Little to no impact if it does not run

CHAPTER 6 · LEARNING 3

Design your systems for it

1: Detect

2: Prioritize

3: Solve

- **It pays off:** production bugs cost more than the patterns
- **Use the toolbox:** after transaction, outbox, idempotency, by scenario and priority
- **Use synergies:** one outbox for every interaction with Zeebe
- **Test and monitor:** prove that it behaves, and get told when it does not

THE ONE SENTENCE TO TAKE HOME

We cannot prevent the problem. We can design for it

No pattern closes the gap. Each one narrows it, at a price you choose

SOURCES

The End

Code-Examples & other sources

github.com/emaarco/distributed-horcruxes

Information from camunda on the topic

- Blog: Navigating technical transactions with Camunda 8 and Spring
- Docs: Message uniqueness
- Video: youtube.com/watch?v=WRR26jJNh68

CAMUNDA CHAPTER MUNICH

Thank you!

MIRAGON

marco.schaeck@miragon.io